Prinsip Perancangan Berorientasi Objek

Mata Kuliah:
CIF61123 – Pola Perancangan (3 SKS)
Pekan ke-2 | Sub-CPMK12
Disusun oleh:
Kelompok Pengelola Mata Kuliah Pola Perancangan

Versi Dokumen: 1.0
Tanggal: 17; Bulan: Agustus; Tahun: 2026
© Fakultas Ilmu Komputer, Universitas Brawijaya



filkom.ub.ac.id

Peta Perkuliahan

Kedudukan pertemuan ini di antara enam belas pekan perkuliahan

01	Konsep dasar & klasifikasi pola
02	**Prinsip perancangan berorientasi objek**
03	Interface, coupling–cohesion, UML
04	SOLID bagian 1 — SRP, OCP, LSP
05	SOLID bagian 2 — ISP, DIP
06	Code smell & refactoring
07	Klaster 1 — kontrol pembuatan objek
08	Ujian Tengah Semester
09	Klaster 2 — komposisi rekursif
10	Klaster 3 — pembungkus antarmuka
11	Klaster 4 — algoritma bervariasi
12	Klaster 5 — ketergantungan longgar
13	Katalog GoF & pedoman pemilihan
14	Studi kasus terpadu bagian 1
15	Studi kasus terpadu bagian 2
16	Ujian Akhir Semester

© Fakultas Ilmu Komputer, Universitas Brawijaya

SubCPMK yang Didukung

Pertemuan ini menyiapkan separuh pertama Sub-CPMK12

12 Sub-CPMK12

Mampu menjelaskan prinsip perancangan berorientasi objek dan memodelkannya dalam diagram UML.

Terkait: CPMK1 — konsep dasar design pattern.

F Bentuk penilaian pekan ini

Formatif — latihan dan diskusi di kelas.

Bobot: 0% (tidak dinilai secara sumatif).

Hasil latihan menjadi bekal LK01 pada pekan ke-3.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Materi Pembelajaran

Tiga pokok bahasan, bersumber dari [1], [2], dan P1

1 Empat pilar berorientasi objek

Abstraksi, enkapsulasi, pewarisan, dan polimorfisme.

2 Komposisi versus pewarisan

Dua cara memakai ulang perilaku beserta konsekuensinya.

3 Studi kasus perbandingan

Dua alternatif rancangan untuk satu kebutuhan yang sama.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Tujuan Pembelajaran

Setelah pertemuan ini mahasiswa diharapkan mampu:

- Menjelaskan abstraksi, enkapsulasi, pewarisan, dan polimorfisme sebagai landasan perancangan berorientasi objek.
- Membedakan pemakaian ulang melalui pewarisan (white-box reuse) dan melalui komposisi (black-box reuse).
- Menganalisis akibat pemilihan pewarisan yang keliru terhadap kemudahan perubahan rancangan.
- Menentukan bagian rancangan yang sebaiknya menggunakan komposisi beserta alasannya.
- Membandingkan dua alternatif rancangan untuk satu kebutuhan yang sama dan memilih yang lebih baik.

© Fakultas Ilmu Komputer, Universitas Brawijaya

01

MATERI KE-1

Empat Pilar Perancangan Berorientasi Objek

Bagian 1 dari 3

© Fakultas Ilmu Komputer, Universitas Brawijaya

Mengapa Prinsip Ini Didahulukan

Pekan 2 sampai 6 membangun fondasi; pola baru dibahas mulai pekan 7

Pola tanpa prinsip

- **Menghafal struktur.** Mahasiswa mengingat gambar kelas, tetapi tidak tahu kapan memakainya.
- **Penerapan mekanis.** Pola dipasang karena terlihat rapi, bukan karena ada masalah yang dipecahkan.
- **Rancangan makin rumit.** Lapisan abstraksi bertambah tanpa manfaat yang sepadan.

Prinsip lebih dahulu

- **Pola sebagai akibat.** Setiap pola GoF adalah penerapan disiplin dari prinsip yang dipelajari pekan ini.
- **Alasan dapat diuji.** Pemilihan pola dapat dinilai benar atau salah berdasarkan prinsip.
- **Bekal seumur karier.** Prinsip tetap berlaku meski katalog pola berubah.

Intinya: pola perancangan adalah prinsip yang dibakukan, bukan resep yang dihafalkan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Empat Pilar

Empat gagasan yang menopang seluruh katalog pola perancangan

A Abstraksi

Menyaring yang penting dan membuang yang tidak relevan bagi pemakai.

Alat: kelas abstrak dan antarmuka.

E Enkapsulasi

Menyembunyikan keadaan internal dan hanya membuka operasi yang disepakati.

Alat: hak akses dan invarian kelas.

P Pewarisan

Menurunkan kontrak dan implementasi dari kelas induk ke kelas turunan.

Alat: extends dan implements.

M Polimorfisme

Satu rujukan bertipe abstrak, banyak perilaku nyata pada saat program berjalan.

Alat: dynamic dispatch.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Abstraksi: Memisahkan Apa dari Bagaimana

Antarmuka adalah bentuk abstraksi yang paling murni

PenyimpanLaporan.java

```java
// Antarmuka menyatakan APA yang bisa dilakukan, bukan BAGAIMANA caranya.
public interface PenyimpanLaporan {
    void simpan(Laporan laporan);
}

// Dua cara berbeda memenuhi kontrak yang sama.
public class PenyimpanBerkas implements PenyimpanLaporan {
    public void simpan(Laporan laporan) { /* tulis ke berkas */ }
}

public class PenyimpanBasisData implements PenyimpanLaporan {
    public void simpan(Laporan laporan) { /* tulis ke tabel */ }
}
```

◆ **Klien cukup tahu kontrak; penggantian cara penyimpanan tidak menyentuh kode klien.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Enkapsulasi: Menjaga Invarian Kelas

Enkapsulasi bukan sekadar menulis private, melainkan menjaga aturan yang harus selalu benar

Melanggar enkapsulasi

```
public class RekeningKuliah {
    public double saldo;   // terbuka bagi siapa pun
}

// Di tempat lain, invarian rusak diam-diam:
rekening.saldo = -750000;
```

Menjaga enkapsulasi

```
public class RekeningKuliah {
    private double saldo;

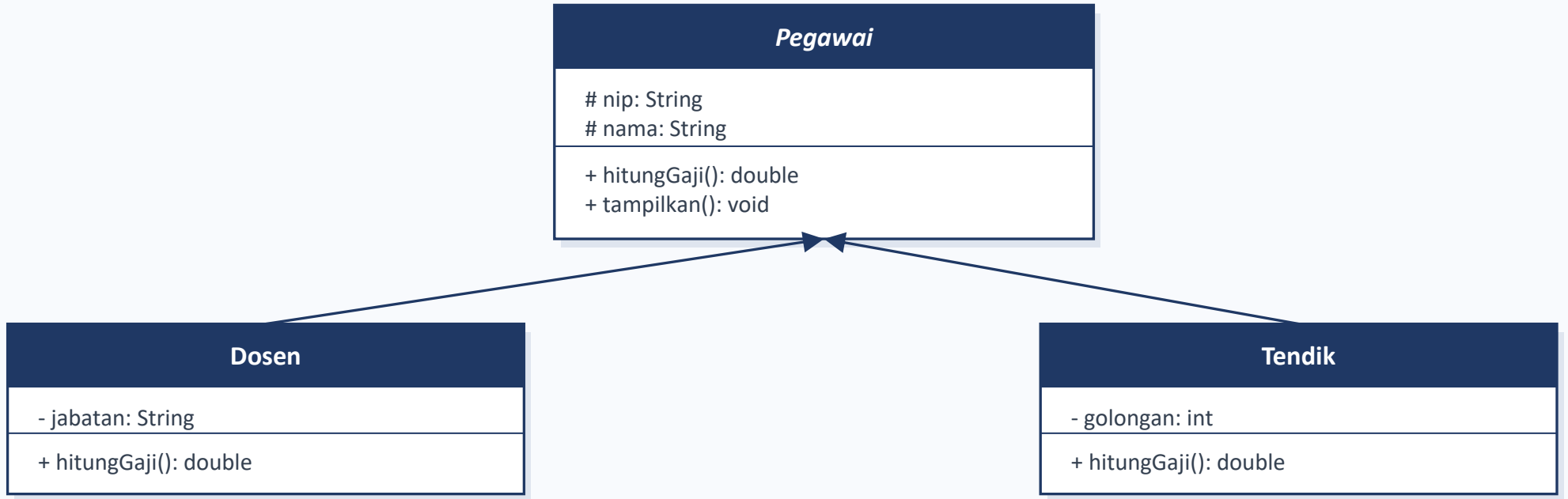
    public void tarik(double jumlah) {
        if (jumlah > saldo) {
            throw new IllegalArgumentException();
        }
        this.saldo -= jumlah;
    }
    public double saldo() { return saldo; }
}
```

- **Aturan “saldo tidak boleh negatif” hanya dapat ditegakkan bila keadaan internal tertutup.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Pewarisan: Menurunkan Kontrak dan Implementasi

Kelas turunan mewarisi tanda tangan operasi sekaligus perilaku bawaan



- **Pewarisan menetapkan hubungan “adalah sebuah” yang bersifat permanen sejak waktu kompilasi.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Polimorfisme: Satu Rujukan, Banyak Perilaku

Metode yang dipanggil ditentukan oleh tipe nyata objek, bukan tipe rujukannya

Polimorfisme.java

```java
public abstract class Pegawai {
    public abstract double hitungGaji();
}

public class Dosen extends Pegawai {
    public double hitungGaji() { return gajiPokok + tunjanganJabatan; }
}
```

```java
public class Tendik extends Pegawai {
    public double hitungGaji() { return gajiPokok + tunjanganGolongan; }
}

// Kode di bawah ini tidak berubah meski jenis pegawai bertambah.
double totalGaji(List<Pegawai> daftar) {
    double total = 0;
    for (Pegawai p : daftar) total += p.hitungGaji();   // dynamic
dispatch
    return total;
}
```

Inilah mekanisme yang kelak membuat hampir seluruh pola GoF bekerja.

© Fakultas Ilmu Komputer, Universitas Brawijaya

02

MATERI KE-2

Komposisi versus Pewarisan

Bagian 2 dari 3

© Fakultas Ilmu Komputer, Universitas Brawijaya

Dua Cara Memakai Ulang Perilaku

Perbedaannya bukan pada sintaks, melainkan pada kapan keterikatan itu terjadi

Pewarisan — white-box reuse

- **Terikat saat kompilasi.** Hubungan induk-turunan tidak dapat diubah ketika program berjalan.
- **Isi induk terlihat.** Turunan bergantung pada rincian internal induk, sehingga enkapsulasi ditembus.
- **Perubahan menular.** Mengubah induk berisiko merusak seluruh turunan.
- **Murah dan langsung.** Tidak perlu menulis kode pendelegasian.

Komposisi — black-box reuse

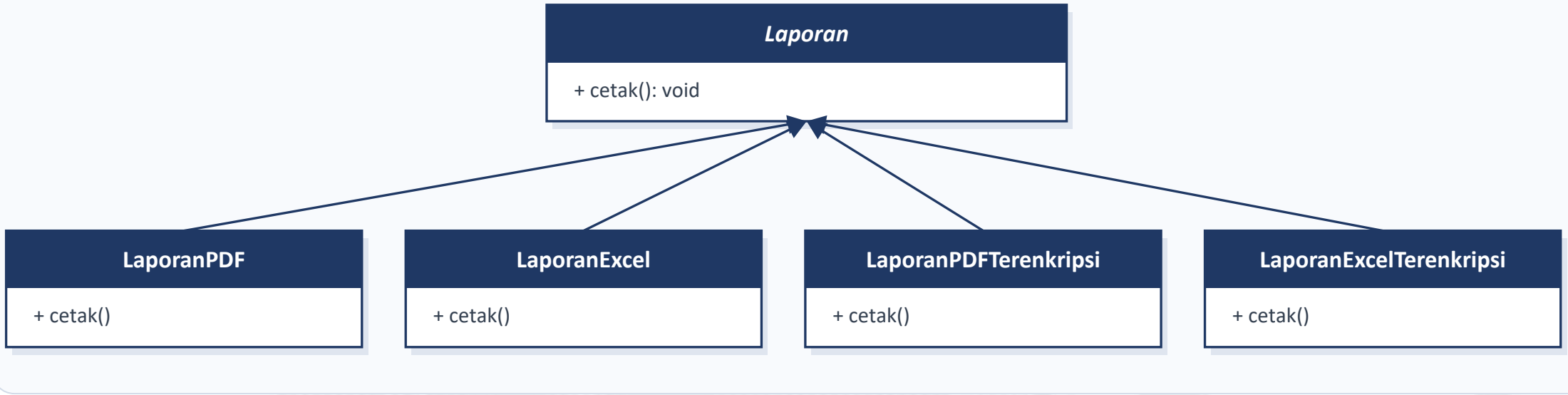
- **Terikat saat berjalan.** Objek yang dipakai dapat diganti kapan saja melalui rujukan.
- **Isi tersembunyi.** Objek pemakai hanya mengenal antarmuka objek yang dipakainya.
- **Perubahan terbendung.** Selama kontrak dipenuhi, penggantian implementasi tidak terasa.
- **Lebih banyak objek.** Rancangan menjadi lebih terpecah dan perlu perakitan.

◆ **Intinya: pilih pewarisan untuk kesamaan jenis, pilih komposisi untuk kesamaan perilaku.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Gejala Salah Pilih: Ledakan Jumlah Subkelas

Dua format kali dua pilihan enkripsi sudah menuntut empat kelas

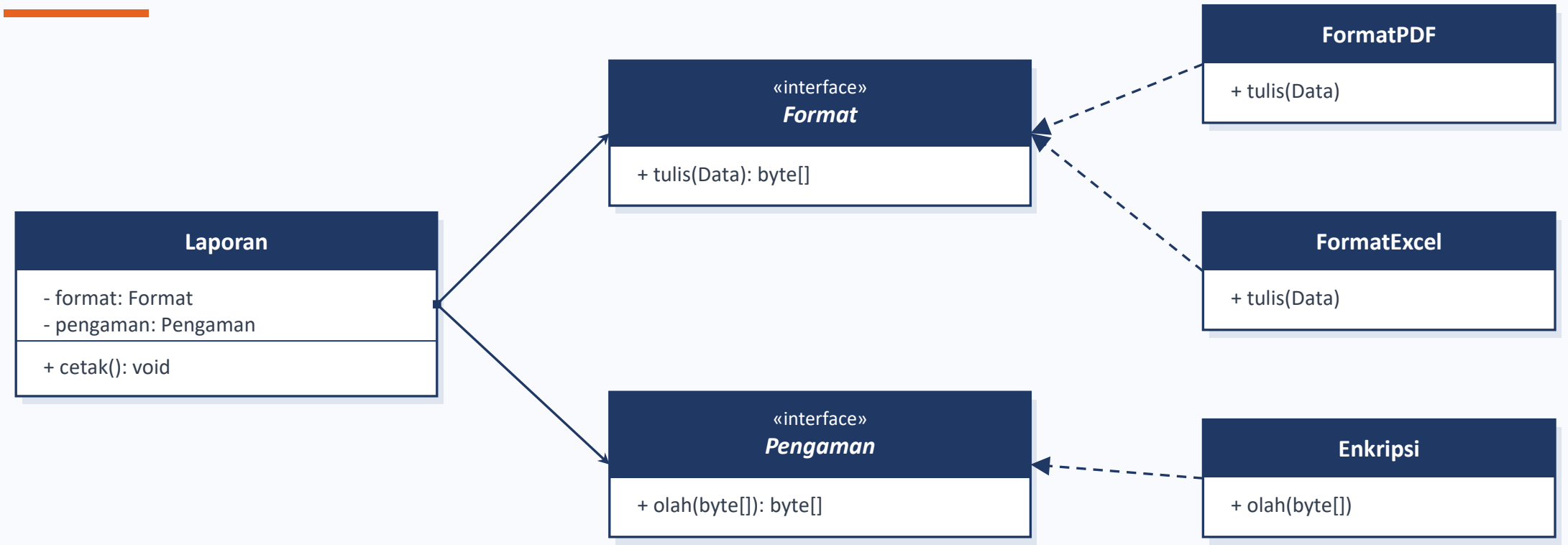


- **Tambah satu format atau satu kemampuan baru, jumlah kelas berlipat — rancangan tidak bertahan.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Rancangan Alternatif dengan Komposisi

Yang berubah-ubah dipisahkan menjadi objek tersendiri



- **Format baru cukup menambah satu kelas; jumlah kombinasi tidak lagi menjadi jumlah kelas.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Perbandingan pada Tingkat Kode

Keterikatan pada waktu kompilasi versus keterikatan pada waktu program berjalan

Pewarisan — kaku

```
class LaporanPDFTerenkripsi extends Laporan {
    public void cetak() {
        byte[] isi = tulisPDF(data);
        byte[] aman = enkripsi(isi);
        kirim(aman);
    }
}

// Ganti format saat program berjalan?
// Tidak bisa — jenis objek sudah tetap.
```

Komposisi — luwes

```
class Laporan {
    private Format format;
    private Pengaman pengaman;

    public void cetak() {
        byte[] isi = format.tulis(data);
        kirim(pengaman.olah(isi));
    }
    public void gantiFormat(Format f) {
        this.format = f;   // saat program berjalan
    }
}
```

◆ **Komposisi memindahkan keputusan rancangan dari waktu kompilasi ke waktu program berjalan.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kapan Pewarisan Masih Merupakan Pilihan yang Tepat

Lima pertanyaan yang dipakai sebelum menulis kata extends

Pertanyaan penguji	Jika jawabannya YA	Jika jawabannya TIDAK
Apakah hubungannya sungguh “adalah sebuah”?	Pewarisan layak dipertimbangkan	Gunakan komposisi
Apakah setiap operasi induk masuk akal bagi turunan?	Pewarisan aman	Gunakan komposisi
Apakah turunan dapat menggantikan induk tanpa kejutan?	Pewarisan memenuhi LSP	Rancangan akan rapuh
Apakah jenisnya tetap sepanjang hidup objek?	Pewarisan cukup	Perlu komposisi
Apakah variasinya lebih dari satu sumbu?	Jumlah subkelas akan meledak	Pewarisan masih terkendali

Pertanyaan ketiga adalah Liskov Substitution Principle, yang dibahas tuntas pada pekan ke-4.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Prinsip Rancangan dari Gang of Four

Dua prinsip pertama pada halaman 18 dan 20 buku Design Patterns [1]

- **Program to an interface, not an implementation.** Klien bergantung pada tipe abstrak, sehingga implementasinya bebas berganti. Dibahas pekan depan.
- **Favor object composition over class inheritance.** Dahulukan komposisi objek; pakai pewarisan hanya ketika hubungan “is-a” benar-benar berlaku.
- **Encapsulate what varies.** Bagian rancangan yang diperkirakan berubah dipisahkan menjadi abstraksi tersendiri.

◆ **Intinya: ketiga prinsip di atas adalah alasan keberadaan hampir seluruh dua puluh tiga pola GoF.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

03

MATERI KE-3

Studi Kasus: Dua Alternatif Rancangan

Bagian 3 dari 3

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kasus: Pengiriman Pemberitahuan Akademik

Bacalah kebutuhan berikut, lalu bandingkan dua rancangan pada slide setelah ini

Kebutuhan

- Sistem mengirim pemberitahuan kepada mahasiswa.
- Saluran saat ini: email dan pesan singkat.
- Akan ditambah: pemberitahuan pada aplikasi telepon genggam.
- Satu pemberitahuan dapat dikirim melalui lebih dari satu saluran.
- Isi pesan disusun berbeda untuk mahasiswa dan dosen.

Yang harus dinilai

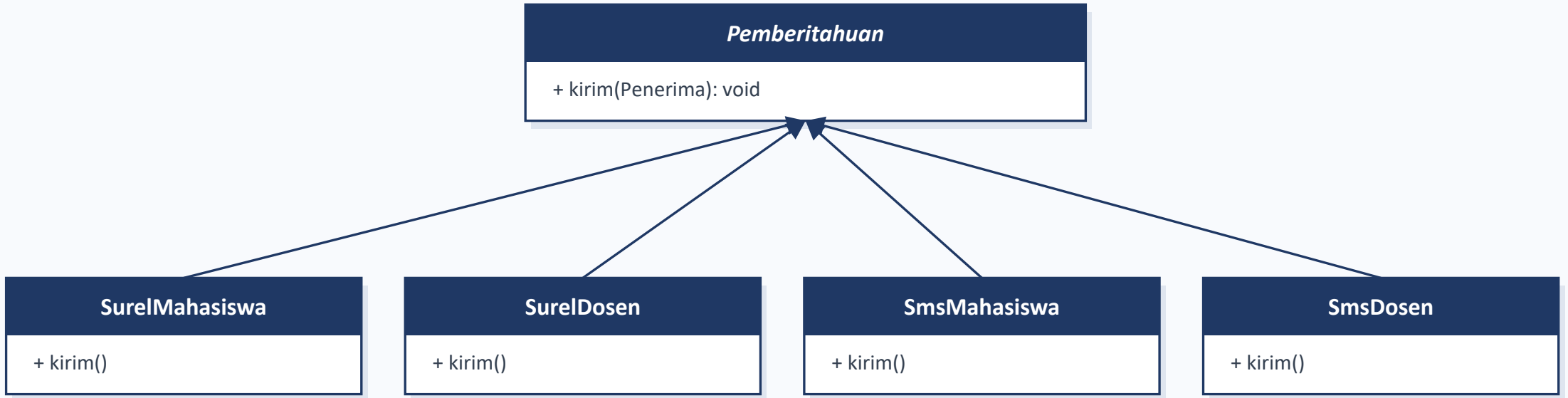
- Sumbu perubahan apa saja yang ada pada kebutuhan ini?
- Bagian mana yang stabil dan bagian mana yang berubah-ubah?
- Berapa banyak kelas yang diperlukan pada tiap alternatif?
- Alternatif mana yang bertahan saat saluran keempat ditambahkan?

◆ **Intinya: terdapat dua sumbu perubahan — saluran pengiriman dan penyusunan isi pesan.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Alternatif A — Semuanya melalui Pewarisan

Setiap kombinasi saluran dan jenis penerima menjadi satu kelas tersendiri

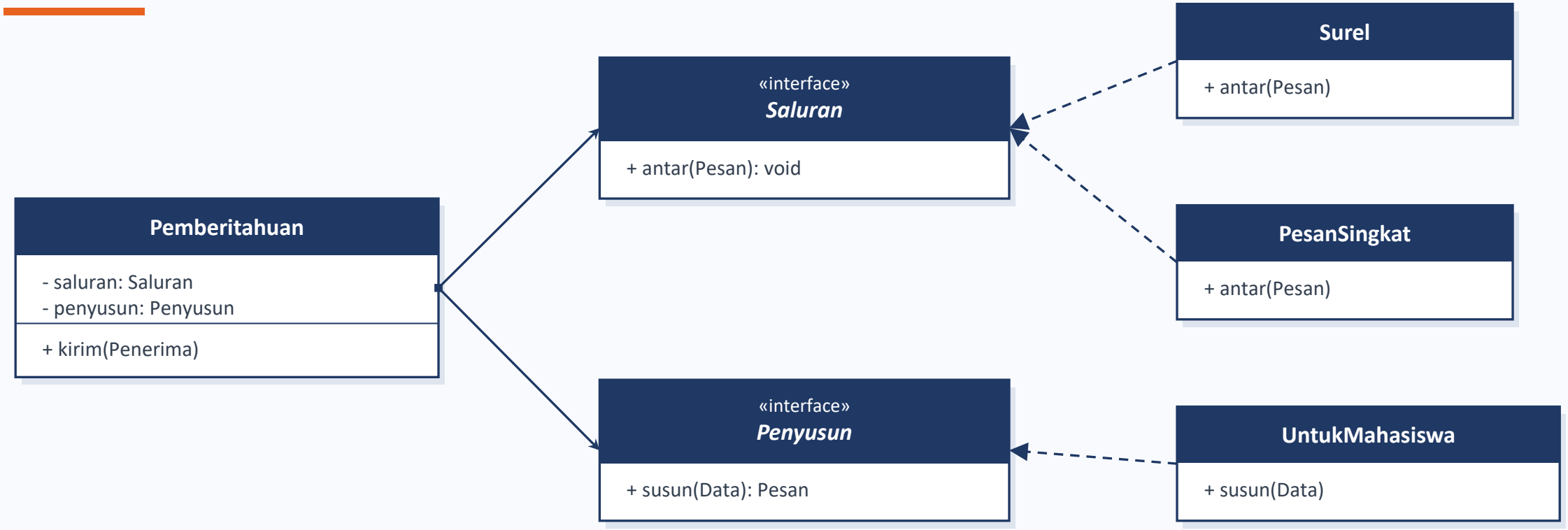


- **Menambah saluran ketiga berarti menambah dua kelas; menambah jenis penerima ketiga menambah tiga lagi.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Alternatif B — Sumbu yang Berubah Dipisahkan

Dua sumbu perubahan menjadi dua abstraksi yang saling bebas



- **Saluran baru cukup satu kelas baru, dan tidak menyentuh kelas Pemberitahuan sama sekali.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Penilaian Kedua Alternatif

Alternatif B baru terlihat unggul ketika kebutuhan bertambah

Kriteria	Alternatif A (pewarisan)	Alternatif B (komposisi)
Jumlah kelas untuk 2 saluran × 2 penerima	5 kelas	6 kelas
Jumlah kelas untuk 4 saluran × 3 penerima	13 kelas	9 kelas
Menambah satu saluran	Menambah kelas sebanyak jenis penerima	Menambah satu kelas saja
Mengganti saluran saat program berjalan	Tidak dapat dilakukan	Cukup mengganti rujukan
Kemudahan pengujian	Perlu objek nyata tiap kombinasi	Cukup objek tiruan tiap antarmuka
Beban perakitan objek	Ringan	Perlu ada yang merakit

Beban perakitan objek pada Alternatif B kelak dijawab oleh creational pattern pada pekan ke-7.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Latihan di Kelas

Kerjakan berpasangan, sepuluh menit, lalu dibahas bersama

1 Sistem penilaian

Nilai akhir dihitung berbeda untuk mata kuliah teori, praktikum, dan skripsi.

Pertanyaan: pewarisan atau komposisi? Sertakan alasannya.

2 Pemutar berkas

Pemutar mendukung berkas MP3 dan WAV, dengan atau tanpa penapis peredam bising.

Pertanyaan: berapa kelas pada tiap alternatif?

3 Kartu akses

Kartu mahasiswa dapat dipakai untuk perpustakaan, laboratorium, dan parkir.

Pertanyaan: sumbu perubahan apa yang perlu diabstraksikan?

© Fakultas Ilmu Komputer, Universitas Brawijaya

Simpulan

- Abstraksi, enkapsulasi, pewarisan, dan polimorfisme adalah empat pilar yang menopang seluruh katalog pola perancangan.
- Pewarisan adalah pemakaian ulang white-box yang terikat pada waktu kompilasi; komposisi adalah pemakaian ulang black-box yang terikat pada waktu program berjalan.
- Ledakan jumlah subkelas adalah gejala paling jelas bahwa pewarisan dipakai untuk menangani lebih dari satu sumbu perubahan.
- Bagian rancangan yang berubah-ubah sebaiknya dipisahkan menjadi abstraksi tersendiri dan dirujuk melalui komposisi.
- Pewarisan tetap tepat ketika hubungan “adalah sebuah” benar-benar berlaku dan turunan dapat menggantikan induk tanpa kejutan.

Pekan depan: program to an interface, coupling dan cohesion, serta pemodelan class diagram dan sequence diagram — sekaligus penjelasan Tugas 1 (LK01).

© Fakultas Ilmu Komputer, Universitas Brawijaya

Pustaka

Gaya pengacuan: Harvard-Anglia

- Gamma, E., Helm, R., Johnson, R. dan Vlissides, J., 1994. Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, hlm. 11–31.
- Refactoring Guru, 2024. Design Patterns. [daring] Tersedia pada: <https://refactoring.guru/design-patterns> [Diakses 5 Juli 2024].
- Freeman, E., Robson, E., Bates, B. dan Sierra, K., 2004. Head First Design Patterns: A Brain-Friendly Guide. Sebastopol, CA: O'Reilly Media, Bab 1.
- Martin, R.C., 2002. Agile Software Development: Principles, Patterns, and Practices. Upper Saddle River, NJ: Prentice Hall.

© Fakultas Ilmu Komputer, Universitas Brawijaya